

Note Systems Smart Contracts Audit Summary

Prepared by Cyfrin

Version 2.0

Head of Security Research

Dacian (Proprietary AI)

Lead Auditors

BengalCatBalu

BenRai

InfiniteSec

October 30, 2026

Contents

1 About Cyfrin 2

2 About This Document 2

3 Disclaimer 2

4 Copyright and License 2

5 Audit Overview 3

6 Audit Scope 3

7 Risk Classification 4

8 Protocol Summary 4

9 Attestation 5

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 About This Document

This document is a public summary of the security audit Cyfrin performed for Note Systems on Smart Contracts. It records the repositories and commits reviewed, the timeline, auditors, in-scope files, and the number and final status of findings by severity. It closes with Cyfrin's attestation of the review's scope and final remediation status.

It is deliberately abbreviated: the technical detail of individual findings is not reproduced here. The complete audit report was delivered privately to Note Systems, who retain it as the full record of the engagement.

3 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

4 Copyright and License

© 2026 Cyfrin Inc. All rights reserved. Material from Note Systems or third parties remains the property of its owners.

This summary may be shared only in complete and unmodified form, so that any copy can be checked against the hash in its attestation. Short quotations with attribution to Cyfrin are permitted, provided they do not misrepresent this summary's scope or results. Except as permitted by applicable law, no part of this summary may otherwise be copied, modified, adapted, sold or sublicensed, or used to create derivative works, without Cyfrin's prior written permission.

Cyfrin and the Cyfrin logo are trademarks of Cyfrin Inc. and may not be used to suggest that Cyfrin has reviewed or endorses anything outside this summary's scope, or to misrepresent its results.

5 Audit Overview

Over the course of 30 days, the Cyfrin team conducted an audit on the Smart Contracts code provided by Note Systems. The findings consist of 1 High, 41 Medium & 101 Low severity issues with the remainder being informational and gas optimizations.

Summary

Project Name	Smart Contracts
Repository	notesystems-project
Commit	cfe6d34eb5c0...
Fix Commit	5db19986691c...
Audit Timeline	Sep 17th - Oct 28th, 2026
Methods	Manual Review, Proprietary AI

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	41	40	1
Low Risk	101	98	3
Informational	34	34	0
Gas Optimizations	6	6	0
Total	183	179	4

6 Audit Scope

The audit scope was limited to:

```
contracts/src/automation/DeskPolicy.sol
contracts/src/automation/NoteAutomation.sol
contracts/src/automation/RollPolicy.sol
contracts/src/core/Governed.sol
contracts/src/core/NoteCore.sol
contracts/src/core/NoteLegs.sol
contracts/src/core/SeriesRegistry.sol
contracts/src/governance/EmissionSchedule.sol
contracts/src/governance/GaugeController.sol
contracts/src/governance/Minter.sol
contracts/src/governance/NoteGovernor.sol
contracts/src/governance/UpgradeGovernor.sol
contracts/src/governance/gauges/DeskGauge.sol
contracts/src/governance/gauges/ERC20RewardGauge.sol
contracts/src/governance/gauges/RewardGauge.sol
contracts/src/governance/gauges/SeriesGauge.sol
contracts/src/governance/gauges/StakingGauge.sol
contracts/src/governance/libraries/TimelockDeployer.sol
contracts/src/governance/libraries/WeekMath.sol
```

```
contracts/src/governance/veNOTE.sol
contracts/src/launch/LaunchParams.sol
contracts/src/libraries/DateTimeLib.sol
contracts/src/libraries/FixedPointMath.sol
contracts/src/oracle/MarketCalendar.sol
contracts/src/oracle/OracleAdapter.sol
contracts/src/oracle/PythProFeed.sol
contracts/src/sale/SaleBase.sol
contracts/src/sale/VestingVault.sol
contracts/src/token/BondDepository.sol
contracts/src/token/Desk.sol
contracts/src/token/NOTE.sol
contracts/src/token/RevenueRouter.sol
contracts/src/token/Treasury.sol
contracts/src/token/libraries/CoreCalls.sol
contracts/src/token/libraries/ExpMath.sol
contracts/src/token/sNOTE.sol
contracts/src/token/upgradeable/GovernedUpgradeable.sol
contracts/src/token/upgradeable/UUPSModule.sol
```

7 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

8 Protocol Summary

Note Systems issues autocallable barrier notes on tokenised US equities on Robinhood Chain. Each series splits into two ERC-1155 legs. COUPON holders deposit USDG and earn a coupon at each observation while the stock stays above the barrier. SHIELD holders deposit Stock Tokens and prefund the coupons to buy principal protection; if the barrier is breached at maturity, their stock goes to the COUPON side. Around the notes sit a USDG vault (the Desk), NOTE and its staking vault sNOTE funded by a fee buyback, a Treasury, a bond depository, and veNOTE governance with two governors: one for parameters behind a 48 hour timelock and one for upgrades behind a 7 day timelock.

8.0.1 Note engine

`NoteCore` runs each series over a fixed schedule of observation closes, from the strike to maturity. If the price is at or above the autocall level at an intermediate observation, the series ends early and COUPON holders redeem principal in USDG. At maturity they also get USDG if the barrier holds. If it is breached, they receive the stock at the strike price `s0` instead, taking the stock's loss. SHIELD holders get their stock back plus any unspent prefund, or the matched USDG on a breach.

`finalizeStrike` fixes `s0` and matches the minimum of four amounts: COUPON deposits, SHIELD stock valued at `s0`, the `notionalCap`, and the notional the prefund can cover. The prefund amount is measured at the lowest prefund-per-stock ratio posted, so every SHIELD unit pays the same. Unmatched amounts become refundable. Legs are transferable, and each transfer settles both parties' accrued coupons first. Payouts are pull-based, come only from per-series escrow, and cannot be paused.

8.0.2 Oracle and calendar

`MarketCalendar` accepts only official US closes as observation times, computing DST on-chain from owner-registered holidays and early closes. `OracleAdapter` takes the last feed round at or before the close, so the price depends only on the close time, not on who calls or when. An optional backup feed is cross-checked for deviation. Unresolved observations are deferred. `PythProFeed` adapts Pyth Pro to Chainlink rounds. It accepts only regular-session prints and blocks prices after an unacknowledged `uiMultiplier` change.

8.0.3 Automation

Strike and observation cranks are permissionless and pay keeper rewards from series fees. User exits also crank due observations, so series progress without keepers. Anyone can `roll` new series from owner-set `RollPolicy` templates, one per underlying per trading-day close. Anyone can also trigger rule-sized Desk fills through `DeskPolicy` for a bounty. `NoteAutomation` batches this work for Chainlink Automation.

8.0.4 Token engine

- `NOTE`: 100M cap, one-shot genesis mint, and whitelisted minters limited per seven-day epoch.
- `sNOTE`: ERC-4626 vault on NOTE. Its rewards stream over seven days so the share price cannot be sandwiched, and withdrawals have a cooldown.
- `Treasury`: holds USDG, stock, COUPON legs and NOTE, marking live legs down by their distance to the barrier. NOTE can be redeemed against that reserve at a haircut, under a daily cap.
- `BondDepository`: sells vesting NOTE for USDG, stock or live legs, at a discount that grows with Desk utilisation.
- `RevenueRouter`: the fee sink. It splits fees between the Treasury and NOTE buybacks: a Dutch auction in the earlier version and a Uniswap pool in the newest. The NOTE it buys goes to `sNOTE`.
- `Desk`: ERC-4626 USDG vault. It takes the COUPON side where SHIELD supply exceeds demand, under caps, and enters with a coupon hurdle. It has a FIFO withdrawal queue and sells settlement stock to the Treasury at the feed price.

8.0.5 Governance

`veNOTE` is a non-transferable, linearly decaying lock of up to four years. It votes in `NoteGovernor` and `UpgradeGovernor`; the latter hard-floors an 8% quorum and a seven-day timelock. veNOTE also sets the `GaugeController` weights that direct the weekly `Minter` emissions to gauges and tighten the Desk's per-underlying caps. `VestingVault` holds NOTE allocations under vesting schedules.

9 Attestation

Cyfrin attests that it performed a security review of the Smart Contracts code provided by Note Systems.

The repositories, commits, review period and findings with their final status are in the Summary and Issues Found tables in Section 5, and the in-scope files in Section 6. This attestation is limited to the documented scope, audited commits and mitigations specifically reviewed by Cyfrin.

Cyfrin publishes this document and its SHA-256 hash at https://github.com/Cyfrin/cyfrin-audit-reports. To check a copy on macOS or Linux, run the following in the folder that contains it; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name. Before running it, check that the URL is Cyfrin's repository.

```
U=https://github.com/Cyfrin/cyfrin-audit-reports/raw/main/reports/sha256
N=2026-10-30-cyfrin-note-systems-smart-contracts-v2.0.pdf
F=$N
H=$(curl -fsSL $U/$N.sha256 | cut -d' ' -f1)
echo "$H *$F" | shasum -a 256 -c
```

Cyfrin's full audit report with all findings provided to Note Systems has the SHA-256 hash and file name below. To check a copy of it, first check this document as above, then run the following in the folder that contains the full report; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name.

```
H=9beec671e8c4e5c26a0ced216e68e1538a11a93d10d93498506cae0aea8f16ac
F=2026-10-30-cyfrin-note-systems-smart-contracts-v2.0-full.pdf
echo "$H *$F" | shasum -a 256 -c
```

On Linux, `sha256sum -c` can be used in place of `shasum -a 256 -c`. On Windows, run `Get-FileHash <file> -Algorithm SHA256` in PowerShell and compare the result with the hash.